

[illegible]

### Mini Carro no Mapa:

0 carro em miniatura no mapa mostra:

- 0Y" N vel da bateria (cor)
- 0Y" Status das travas
- 0Y" Localiza  o GPS
- 0Y...i. Marcha atual
- 0Y'i Far is acesos/apagados

---

## Tipos de Conex o

0 BYD Connect suporta \*\*5 tipos de conex o\*\* para se comunicar com o carro:

| Tipo      |  cone | Uso             | Porta Padr o |
|-----------|-------|-----------------|--------------|
| HTTP/REST | 0Y    | APIs JSON       | 8080         |
| SSH       | 0Y- i | Terminal direto | 22           |
| WebSocket | 0Y    | Tempo real      | 8080         |
| MQTT      | 0Y"i  | IoT/MQTT        | 1883         |
| Bluetooth | 0Y"   | OBD-II          | -            |

---

## 1. Configura  o HTTP/REST API

### Quando usar?

- Servidor backend exp e API REST
- Retorno de dados em JSON
- Integra  o com sistemas web

### Par metros:

| Campo   | Descri  o            | Exemplo       |
|---------|----------------------|---------------|
| Host/IP | Endere o do servidor | 192.168.1.100 |
| Porta   | Porta do servidor    | 8080          |
| SSL/TLS | Usar HTTPS           | true/false    |
| Usu rio | Autentica  o b sica  | admin         |
| Senha   | Autentica  o b sica  | ****          |
| API Key | Chave de API         | xyz123        |

### Servidor Flask (Python):

```
```python
from flask import Flask, jsonify, request

app = Flask(__name__)

@app.route('/api/status', methods=['GET'])
def get_status():
    return jsonify({
        'status': {
            'isOnline': True,
            'batteryLevel': 78,
            'gear': 'P',
            'isLocked': True,
            'speed': 0,
            'range': 420,
            'temperature': 25,
            'acIsOn': False,
            'trunkIsOpen': False,
            'windowsState': 'closed',
            'lightsAreOn': False,
            'engineRunning': False,
            'mirrorsFolded': False
        },
        'battery': {
            'voltage': 356.4,
            'current': -15.3,
            'temperature': 28,
            'capacity': 82,
            'cycles': 127,
            'health': 98
        },
        'odometer': 12580
    })
```

```

    })

@app.route('/api/command/<command>', methods=['POST'])
def send_command(command):
    data = request.json or {}
    # Processar comando
    return jsonify({'success': True, 'command': command})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080)
...

### Endpoints da API:

| Endpoint | Método | Descrição |
|-----|-----|-----|
| `/api/status` | GET | Status básico |
| `/api/car/status` | GET | Status completo |
| `/api/command/{cmd}` | POST | Enviar comando |
| `/api/location` | GET | GPS do carro |

---

## 2. Configuração SSH

### Quando usar?
- Acesso direto ao terminal Linux do carro
- Scripts customizados
- Debugging avançado

### Parâmetros:

| Campo | Descrição |
|-----|-----|
| Host/IP | Endereço SSH |
| Porta | Padrão: 22 |
| Usuário | Login SSH |
| Senha | Senha SSH |

### Comandos SSH:

```bash
# Status do carro
$ car status
{"batteryLevel":78,"isLocked":true,"speed":0}

# Travar/destravar
$ car lock on
$ car lock off

# Faróis
$ car lights on
$ car lights off

# Ar condicionado
$ car ac on
$ car ac off

# Porta-malas (0-100%)
$ car trunk 50

# Vidros
$ car windows closed
$ car windows half
$ car windows open

# Localização
$ car location
{"lat":-23.5505,"lon":-46.6333,"address":"São Paulo, SP"}
...

## 3. Configuração WebSocket

### Quando usar?
- Atualizações em tempo real

```

- Alta frequência de dados
- Apps interativos

### Servidor WebSocket (Node.js):

```
```javascript
const WebSocket = require('ws');
const wss = new WebSocket.Server({ port: 8080 });

wss.on('connection', (ws) => {
  console.log('Carro conectado');

  ws.on('message', (message) => {
    const data = JSON.parse(message);
    console.log('Comando:', data.command);

    // Processar e responder
    ws.send(JSON.stringify({
      type: 'response',
      success: true,
      data: { /* status atualizado */ }
    }));
  });

  // Enviar status a cada 5 segundos
  setInterval(() => {
    ws.send(JSON.stringify({
      type: 'status_update',
      data: getCarStatus()
    }));
  }, 5000);
});

function getCarStatus() {
  return {
    batteryLevel: 78,
    isLocked: true,
    speed: 0
  };
}
```
```

### Mensagens WebSocket:

```
```json
// Cliente â†’ Servidor
{
  "command": "lock",
  "params": {"locked": true},
  "timestamp": "2024-01-15T10:30:00Z"
}

// Servidor â†’ Cliente
{
  "type": "status_update",
  "data": {
    "isLocked": true,
    "batteryLevel": 78
  }
}
```
```

---

## 4. Configuração MQTT

### Quando usar?

- Dispositivos IoT
- Alta eficiência de banda
- Arquitetura publish/subscribe

### Tópicos MQTT:

| Tópico        | Direção | Descrição         |
|---------------|---------|-------------------|
| /byd/status   | â†’     | Status do veículo |
| /byd/location | â†’     | Coordenadas GPS   |

[illegible]



```
**Causas:**
- Credenciais incorretas
- Token expirado
- SSL não configurado

**Soluções:**
1. Verifique usuário/senha
2. Regenere API Key
3. Ative/desative SSL

### ❸ App Fecha ao Conectar

**Causas:**
- Dados mal formatados
- JSON inválido
- NullPointerException

**Soluções:**
1. Verifique formato JSON
2. Teste API com Postman
3. Reinicie o app

---

## ❷ Suporte

### Logs de Debug

Para debugar, ative no app:
- Menu ❶ Configurações ❶ Debug
- Ativar logs de rede

### Teste de Conexão

```bash
# Testar HTTP
curl http://IP:PORTA/api/status

# Testar WebSocket
wscat -c ws://IP:PORTA

# Testar MQTT
mosquitto_sub -h IP -t byd/status
```

---

## ❶ Resumo Rápido

| Etapa | Ação |
|-----|-----|
| 1 | Abra a aba "Conexão" |
| 2 | Selecione o tipo (HTTP/SSH/etc) |
| 3 | Digite o IP do servidor |
| 4 | Clique em "CONECTAR" |
| 5 | Aguarde confirmação verde |
| 6 | Dados aparecem no Dashboard |

---

*Documento gerado em 2024*
*BYD Connect App v1.0*
*Manual de Conexão e Integração*
```